From Training to Serving: Model Formats, Serialization, and Inference Engineering

A First-Principles Examination of the LLM Inference Stack

Bernard

May 24, 2026

Abstract

This article examines the engineering stack that bridges LLM training and serving, from first principles. We cover what training produces, how models are serialized into portable formats, the design decisions embedded in each format, and how serving engines reconstruct and execute models at scale. The focus is on the "why" behind each design choice: alignment requirements, memory access patterns, fragmentation, and the fundamental tension between generality and performance.

Contents

Introduction

When a large language model finishes training, what exactly is produced? The naive answer—"a bunch of numbers"—misses the extraordinary engineering depth in model serialization and serving. Between training and inference lies a pipeline of format conversions, quantisation passes, engine compilation, and runtime optimisation that can multiply or divide throughput by an order of magnitude.

This article traces that pipeline from end to end:

1. What training produces (checkpoints, optimizer states, metadata)
2. How models are serialized (format design, alignment, memory mapping)
3. How formats are converted (the hidden complexity of weight rearrangement)
4. How serving engines load and execute models (batching, caching, parallelism)
5. The production deployment layer (scaling, monitoring, reliability)

Throughout, we emphasise *first principles*: memory bandwidth is the bottleneck, not compute. Formats are designed around access patterns, not storage efficiency. Serving is about hiding latency, not reducing it.

What Training Produces

The Checkpoint

Training a large language model produces a *checkpoint*: a snapshot of the model state at a given training step. A complete checkpoint contains:

- **Model parameters:** the weight matrices and bias vectors defining every layer.
- **Optimizer state:** for Adam, this includes first moment estimates and second moment estimates—effectively two additional copies of every parameter.
- **Scheduler state:** current learning rate, step count, epoch.
- **RNG state:** random number generator seeds for reproducibility.
- **Trainer metadata:** loss history, data loader state, configuration.

Definition 2.1 (Memory Usage During Training)**.** For a model with N parameters in FP32:

- Parameters: $4N$ bytes
- Gradients: $4N$ bytes
- Adam optimizer states: $8N$ bytes (momenta m_t and v_t)
- Activations (forward pass): varies, $O(L \cdot B \cdot S \cdot d)$

Total training memory: approximately $16N$ bytes for the model state alone, plus activations that can dominate for long sequences.

For a 70B model in BF16 training:

- Parameters (BF16): 140 GB
- Gradients (BF16): 140 GB
- Optimizer states (FP32): 560 GB (two FP32 copies per parameter)
- Total model state: 840 GB across all GPUs

ZeRO Optimization and Sharded Checkpoints

DeepSpeed ZeRO partitions optimizer states, gradients, and parameters across GPUs. This produces *sharded checkpoints*:

- **ZeRO Stage 1:** optimizer states partitioned; parameters and gradients replicated.
- **ZeRO Stage 2:** optimizer states + gradients partitioned.
- **ZeRO Stage 3:** optimizer states + gradients + parameters partitioned.

Stage 3 reduces memory per GPU by approximately $N_{\text{GPU}}\times$, but means no single GPU holds the full model. Checkpoint saving requires an all-gather to consolidate, or saving the sharded state with a metadata file describing how to reconstruct.

Listing 1: ZeRO checkpoint directory structure

```
checkpoint-100/
|-- zero_pp_rank_0_mp_rank_00_optim_states.pt
|-- zero_pp_rank_1_mp_rank_00_optim_states.pt
|-- zero_pp_rank_2_mp_rank_00_optim_states.pt
|-- zero_pp_rank_3_mp_rank_00_optim_states.pt
|-- latest_checkpointed_iteration.txt
|-- zero_to_fp32.py # reconstruction script
+-- ds_config.json # training configuration
```

What Matters for Inference

The inference engine only needs the model parameters—not gradients, not optimizer states, not RNG seeds. This immediately suggests the first conversion: strip the checkpoint down to bare weights.

But even "bare weights" carry metadata:

- **Architecture config:** number of layers, hidden dimension, head count, vocab size, activation function, normalization type, RoPE base frequency.
- **Tensor layout:** which axes are contiguous, how parameters are grouped.
- **Dtype:** the numerical format of each tensor (BF16, FP16, FP32, INT8).

The separation of weights from optimizer state is the first design decision in the inference stack: *do not ship the optimizer state.*

Model Serialization Formats

The Problem

Serializing 140 GB of parameters (70B model, BF16) requires solving several engineering problems:

1. **Fast deserialization:** loading 140 GB should not take minutes.
2. **Zero-copy reads:** if possible, map the file directly into memory without copying.
3. **Alignment:** tensors must be aligned to GPU memory boundaries (typically 256 bytes for H100).
4. **Safety:** loading a file should not execute arbitrary code.
5. **Portability:** the file should work across platforms (endianness, CUDA versions).

Each format makes different trade-offs along these axes.

PyTorch Pickle Format (.pt, .pth, .bin)

The original format shipped with PyTorch. It is a Python pickle of a dictionary mapping string names to tensors.

Listing 2: Saving and loading in PyTorch

```
# Save
torch.save({
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optim.state_dict(),
    "step": step,
    "loss": loss,
```

```
}, "checkpoint.pt")

# Load
checkpoint = torch.load("checkpoint.pt", map_location="cpu")
model.load_state_dict(checkpoint["model_state_dict"])
```

3.2.1 How It Works

Internally, PyTorch's serialization:

1. Pickles the Python object graph (nested dictionaries of tensors).
2. Each tensor is serialized as a header (shape, dtype, stride) + raw data.
3. Raw tensor data is written contiguously using `tensor.numpy().tofile()`.
4. A file-level key records offsets for each tensor's data.

3.2.2 Problems

- **Security:** pickle executes arbitrary Python code during deserialization. Loading an untrusted checkpoint is equivalent to `eval(open(file).read())`.
- **Speed:** pickle parsing is single-threaded and slow for large files.
- **Memory:** `torch.load` deserializes into CPU memory first, then copies to GPU. At minimum, one full copy of the model exists in CPU RAM during loading.
- **No alignment guarantees:** tensor data starts at arbitrary file offsets, which may not align to GPU memory pages.
- **Difficult to partially load:** to load one tensor, you must parse the entire pickle.

Despite these flaws, PyTorch pickle remains universal because it is the native format and supports arbitrary Python objects (tied weights, shared buffers).

SafeTensors (.safetensors)

SafeTensors was designed by HuggingFace to address pickle's security and performance problems. It is the recommended format for model distribution on the HuggingFace Hub.

3.3.1 Format Specification

The format is deliberately simple:

Listing 3: SafeTensors binary layout

```
+------------------+
| Header (JSON) | <- 8-byte aligned length prefix
+------------------+
| Tensor 0 data | <- 64-byte aligned
+------------------+
| Tensor 1 data | <- 64-byte aligned
+------------------+
| ... |
+------------------+
```

The header is a JSON object (preceded by an 8-byte little-endian length) mapping tensor names to metadata:

Listing 4: SafeTensors header structure

```
{
  "lm_head.weight": {
    "dtype": "BF16",
    "shape": [32000, 4096],
```

```
    "data_offsets": [0, 262144000]
  },
  "model.layers.0.self_attn.q_proj.weight": {
    "dtype": "BF16",
    "shape": [4096, 4096],
    "data_offsets": [262144000, 295698432]
  },
  ...
}
```

Key design decisions:

- **No code execution:** the header is JSON, parsed safely by any JSON library.
- **Alignment:** each tensor's data starts at a 64-byte aligned offset. This matches GPU memory alignment requirements.
- **Zero-copy mmap:** because the file format is flat and aligned, the file can be memory-mapped and tensors can point directly into the mmap region without copying.
- **Parallel loading:** multiple tensors can be loaded in parallel—the header provides all offsets upfront, so no stateful parsing is needed.
- **Endian-aware:** the header stores byte length; the reader checks endianness.

Theorem 3.1 (SafeTensors Loading Complexity)**.** Loading a SafeTensors file is $O(1)$ in parsing time (independent of model size), plus $O(\text{data})$ in memory mapping. Compare with pickle, which is $O(N_{\text{tensors}})$ for unpickling plus $O(\text{data})$ for data copy.

3.3.2 Limitations

- No support for shared tensors (where multiple names point to the same memory).
- No compression—the file is a direct byte-for-byte dump of GPU-ready data.
- No support for arbitrary Python objects (requires separate config files).

GGUF (.gguf)

GGUF (GGML Universal Format) was designed for llama.cpp. It targets CPU inference on consumer hardware, where quantisation and memory mapping are critical.

3.4.1 Format Architecture

GGUF is a versioned, extensible binary format:

Listing 5: GGUF binary layout

```
+---------------------------+
| GGUF magic (0x46554747) | <- "GGUF" in ASCII
| Version (uint32_t) |
| Tensor count (uint64_t) |
| Metadata key-value pairs |
+---------------------------+
| Tensor info table |
| (name, n_dims, dtype, |
| offset, dimensions) |
+---------------------------+
| Aligned padding |
+---------------------------+
| Tensor 0 data | <- GGML_MEM_ALIGN (32-byte) aligned
+---------------------------+
| Tensor 1 data |
+---------------------------+
```

```
| ... |
+---------------------------+
```

Key differences from SafeTensors:

- **Multiple quantisation types:** GGUF encodes the quantisation type per-tensor (Q4_0, Q4_K_M, Q5_1, Q8_0, etc.), not just the dtype.
- **Metadata first:** model hyperparameters (vocab size, RoPE theta, layer count) are embedded in the file, eliminating the need for a separate `config.json`.
- **Vocabulary embedding:** the tokenizer vocabulary is included directly, making the file fully self-contained.
- **GGML alignment:** tensor data is aligned to 32 bytes (matching GGML's memory allocation), not 64 bytes (SafeTensors) or 256 bytes (CUDA).
- **Extensible via KV metadata:** arbitrary key-value pairs can be added without format breakage.

3.4.2 Quantisation Encoding

GGUF's quantisation types encode the block structure directly:

Type	Bits/weight	Block size	Block structure	Use case
Q4_0	4.5	32	1 FP16 scale + 32 × 4-bit	Fastest 4-bit
Q4_1	5.0	32	2 FP16 scales + 32 × 4-bit	Better quality
Q4_K_M	4.5	256	Importance-weighted superscalar	Default 4-bit
Q5_0	5.5	32	1 FP16 scale + 32 × 5-bit	Fast 5-bit
Q5_K_M	5.5	256	Importance-weighted superscalar	Default 5-bit
Q8_0	9.0	32	1 FP16 scale + 32 × 8-bit	Near-lossless

The `_K_M` variants ("K-quant") use a superscalar block structure: within each 256-weight block, weights are grouped into sub-blocks of 16 or 32, each with its own scale. The importance weighting allocates more bits to weights that affect the output more.

3.4.3 Meta-Information in GGUF

Listing 6: GGUF metadata keys (excerpt)

```
{
  "general.architecture": "llama",
  "llama.context_length": 8192,
  "llama.embedding_length": 4096,
  "llama.feed_forward_length": 11008,
  "llama.attention.head_count": 32,
  "llama.attention.head_count_kv": 8,
  "llama.rope.freq_base": 10000.0,
  "llama.block_count": 32,
  "llama.vocab_size": 32000,
```

```
  "tokenizer.ggml.model": "gpt2",
  "tokenizer.ggml.tokens": ["the", "a", "an", ...]
}
```

ONNX (.onnx)

ONNX (Open Neural Network Exchange) is an interchange format, not a deployment format. It represents the computation graph as a protobuf-serialized graph.

- **Strengths:** hardware vendor support (NVIDIA, Intel, AMD, Apple), graph optimisation passes, operator fusion, quantisation via QOperator/QLinearConv.
- **Weaknesses:** large file size (protobuf overhead), slow loading, awkward dynamic shapes, poor support for transformer-specific operations (GQA, RoPE, causal mask).

ONNX is increasingly supplanted by framework-specific formats (TensorRT engines, CoreML models, MPS graphs) for production deployment.

Comparison Table

Property	SafeTensors	GGUF	PyTorch	ONNX
Safe to load	Yes	Yes	No (pickle)	Yes
Zero-copy mmap	Yes	Yes	No	No
Tensor alignment	64-byte	32-byte	None	None
Quantisation	No	Yes (per-block)	No	QOperator
Tokenizer embedded	No	Yes	No	No
Config embedded	No	Yes	No	Partial
Parallel load	Yes	Yes	No	No
GPU direct load	Partial	CPU-first	No	No
Compression	None	None	None	None
Extensible	No	KV pairs	Yes	Yes

The Conversion Pipeline

Converting a PyTorch checkpoint to an inference format is not a simple dtype cast. Several hidden complexities arise.

Weight Rearrangement

Different inference engines expect weights in different layouts:

- **PyTorch:** row-major, any layout.
- **SafeTensors:** row-major, aligned at tensor boundaries.
- **GGUF:** row-major per-tensor, quantisation blocks at 32 or 256 elements.
- **TensorRT-LLM:** requires QKV weights fused into a single matrix, KV heads permuted for GQA, and layernorm weights merged.
- **vLLM:** expects weights compatible with the attention backend (FlashInfer, FlashAttention, or custom CUDA kernels).

The most common rearrangement is *QKV fusion*:

Listing 7: QKV weight fusion for inference

```
# PyTorch stores separate Q, K, V projections
q_weight = state_dict["self_attn.q_proj.weight"] # [d, d]
k_weight = state_dict["self_attn.k_proj.weight"] # [d, d_kv]
v_weight = state_dict["self_attn.v_proj.weight"] # [d, d_kv]

# TensorRT-LLM fuses them into a single GEMM
qkv_weight = torch.cat([
    q_weight,
    k_weight,
    v_weight,
], dim=0) # [d + 2*d_kv, d]

# Additional permutation for GQA head reordering
n_heads = 32
n_kv_heads = 8
# ... permute k_weight rows to match GQA group structure
```

Dtype Conversion

During conversion, weights are typically cast from training dtype to inference dtype:

Conversion	Rationale
FP32 → BF16	Standard inference precision
FP32 → FP16	Older GPUs without BF16 support
BF16 → FP8	H100 tensor core support
BF16 → INT4	Memory-constrained deployment

Each conversion has a quality impact. For FP16/BF16, quality loss is negligible. For INT4, perplexity typically increases by 0.5–2 points depending on the quantisation method.

LayerNorm/RMSNorm Fusion

During training, LayerNorm weights and biases are stored separately. Inference engines fuse them into the preceding or following GEMM to reduce kernel launches:

$$\text{Fused: } y = \frac{Wx + b - \mu}{\sigma} \cdot \gamma + \beta \tag{1}$$

This fusion is a constant-fold at conversion time, baked into the engine's compiled graph.

RoPE Precomputation

Many engines precompute RoPE cos/sin tables and embed them in the serialized engine:

Listing 8: RoPE table precomputation

```
def precompute_rope(max_seq_len, dim, theta=10000.0):
    inv_freq = 1.0 / (theta ** (torch.arange(0, dim, 2) / dim))
    t = torch.arange(max_seq_len)
    freqs = torch.outer(t, inv_freq)
    return torch.stack([torch.cos(freqs), torch.sin(freqs)])
```

Precomputation avoids regenerating these tables on every inference call—a small optimisation that compounds over millions of tokens.

Inference Serving Architecture

The Fundamental Problem

Serving an LLM means: given a stream of incoming requests, maximise throughput (tokens/second) while minimising latency (time-to-first-token, inter-token latency). These goals conflict:

- **Throughput** improves with larger batches (amortise GPU costs).
- **Latency** degrades with larger batches (queuing, memory contention).

The solution is *continuous batching*, but its implementation requires solving several sub-problems.

Request Lifecycle

Each request goes through the following stages:

1. **Pre-fill:** process the input prompt, generate the first output token. Compute-bound (attention over the full prompt).
2. **Decode:** generate subsequent tokens one at a time. Memory-bound (attention over KV cache).
3. **Done:** return the response and free resources.

The transition from pre-fill to decode changes the compute profile dramatically. Pre-fill saturates GPU compute units; decode is bottlenecked by memory bandwidth.

Continuous Batching

Continuous batching (also called "in-flight batching" or "iteration-level scheduling") admits new requests as soon as a slot opens, rather than waiting for all requests in a batch to finish.

Algorithm 5.1 (Continuous Batching)**.** 1. Maintain a scheduling queue of pending requests.
2. At each iteration:
   (a) Collect all active decoding sequences.
   (b) If GPU memory allows, admit 1+ new pre-fill requests from the queue.
   (c) Run one forward pass on the batch (mixed pre-fill + decode).
   (d) Return completed responses; update scheduling state.

5.3.1 Mixed Pre-fill and Decode

Running pre-fill (compute-bound) and decode (memory-bound) in the same batch is inefficient because they bottleneck different resources. Modern schedulers handle this by:

- **Separate scheduling:** dedicate some GPU capacity to pre-fill, some to decode.
- **Pre-fill prioritisation:** run pre-fill first (to reduce TTFT), then decode.
- **Chunked pre-fill:** split long prompts into chunks, interleaving with decode.

PagedAttention and Block Management

The KV cache is the dominant memory consumer during serving. For a 70B model serving 100 concurrent requests at 8192 tokens each:

$$M_{\text{KV}} = 2 \cdot 80 \cdot 8 \cdot 8192 \cdot 128 \cdot 2 \cdot 100 \approx 268 \text{ GB}. \tag{2}$$

This exceeds the HBM of most GPU configurations. Efficient management is critical.

5.4.1 The Fragmentation Problem

Naive KV cache allocation reserves a contiguous block of memory for each sequence, worst-case length. This causes:

- **Internal fragmentation:** reserved but unused capacity within each sequence (most sequences are shorter than the maximum).
- **External fragmentation:** freed blocks of different sizes scatter across memory.
- **Reservation overhead:** memory must be reserved upfront, limiting concurrency.

5.4.2 PagedAttention (vLLM)

PagedAttention (Kwon et al., 2023) solves fragmentation by allocating the KV cache in fixed-size pages (blocks):

Definition 5.1 (KV Block)**.** A KV block stores the keys and values for a fixed number B of tokens (typically $B = 16$). Blocks are the unit of allocation and deallocation.

Listing 9: Block table for a single sequence

```
Sequence: [token 0..15] [token 16..31] [token 32..47] [token 48..55]
Blocks: [block 7] [block 92] [block 31] [block 44]

# Logical-to-physical mapping via block table:
block_table = {0: 7, 1: 92, 2: 31, 3: 44}
```

Key properties:

- **No internal fragmentation:** unused slots within a block are at most $B - 1$ tokens.
- **No external fragmentation:** all blocks are the same size; any free block fits any slot.
- **Copy-on-write:** for parallel sampling (beam search), blocks shared across samples are COW-protected.
- **Prefix caching:** common prefixes (system prompts) share blocks across requests.

Theorem 5.1 (PagedAttention Memory Savings)**.** For S sequences with average length $\bar{L}$ and block size B, PagedAttention uses:

$$M_{\text{PA}} = S \cdot \lceil \bar{L}/B \rceil \cdot B \cdot (2 \cdot d_k \cdot p) \tag{3}$$

compared to $S \cdot L_{\max} \cdot (2 \cdot d_k \cdot p)$ for contiguous allocation. When $\bar{L} \ll L_{\max}$, savings are proportional to $L_{\max}/\bar{L}$.

5.4.3 Block Manager Architecture

The vLLM block manager implements:

- **Free pool:** a list of available blocks.
- **Allocator:** assigns blocks to new tokens, handles COW.
- **Swapper:** moves blocks between CPU and GPU memory when GPU memory is exhausted (offloading).
- **Evictor:** selects blocks to evict when memory is full (LRU or prefix-aware).

Prefix Caching

When multiple requests share a common prefix (system prompt, few-shot examples), their KV cache entries for that prefix are identical. PagedAttention's block structure enables *automatic prefix caching*:

- Compute the KV cache for the prefix once.
- All subsequent requests with the same prefix reuse those blocks.

- Block hashing detects prefix matches in $O(1)$ per block.
- Common fragments (e.g., "`The capital of France is`") are cached across unrelated requests.

Theorem 5.2 (Prefix Cache Hit Ratio)**.** For a deployment with K distinct prefixes (system prompts), S requests, and average prefix length L_p, the cache hit ratio is:

$$\text{hit}\% = 1 - \frac{K}{S} \quad \text{when } S \gg K. \tag{4}$$

Each prefix's blocks are computed once and reused S/K times on average.

Speculative Decoding in Production

Speculative decoding accelerates generation by using a small draft model. In production, several engineering considerations arise:

- **Draft model selection:** the draft model must be small enough to run quickly but accurate enough to achieve high acceptance rates. Typical choice: 5–10% of the target model's size.
- **Draft length:** longer drafts increase verification cost. Optimal draft length balances acceptance rate against the risk of wasted computation.
- **Verification efficiency:** the target model's forward pass must handle variable-length batches (draft tokens + ground truth tokens).
- **Tree attention:** for speculative decoding with multiple candidates (Medusa-style), tree-structured attention processes all candidates in one forward pass.

Tensor Parallelism and Model Sharding

For models larger than a single GPU's memory, parameters are sharded across GPUs:

- **Row-wise sharding:** each GPU holds a subset of rows; all-reduce aggregates results.
- **Column-wise sharding:** each GPU holds a subset of columns; no communication needed for forward pass.
- **2D sharding:** combine row and column sharding for very large models.

Theorem 5.3 (All-Reduce Cost)**.** For tensor parallelism across P GPUs, the all-reduce communication volume per transformer layer is:

$$V_{\text{comm}} = 2 \cdot P \cdot B \cdot S \cdot d \quad \text{bytes}, \tag{5}$$

where B is batch size, S is sequence length, d is hidden dimension. Each all-reduce takes $2(P-1)/P \cdot V_{\text{comm}}$ bandwidth on a ring topology.

Serving Framework Architecture

vLLM

vLLM (Kwon et al., 2023) is the most widely adopted open-source LLM serving engine.

6.1.1 Architecture

Listing 10: vLLM architecture overview

```
Request
  |
  v
```

```
[HTTP Frontend] <- OpenAI-compatible API
  |
  v
[Scheduler] <- Continuous batching, prefix caching,
| block management
| |
| v
| [Block Manager] <- PagedAttention blocks
| |
| v
| [Worker Pool] <- GPU workers (tensor parallelism)
| | | |
| v v v
| [GPU 0] [GPU 1] [GPU 2]
|
v
Response
```

Key subsystems:

- **LLMEngine:** orchestrates scheduling, block management, and distributed execution.
- **ModelRunner:** executes the model forward pass, handling weight loading and attention backends.
- **Attention backend:** pluggable (FlashAttention, FlashInfer, PagedAttention).
- **Weight loading:** reads SafeTensors, supports quantisation (AWQ, GPTQ, FP8).

6.1.2 Attention Backend Abstraction

vLLM abstracts attention computation behind a backend interface:

Listing 11: Attention backend interface (simplified)

```
class AttentionBackend:
    def forward(
        self,
        query: Tensor, # [num_tokens, num_heads, head_size]
        key: Tensor,
        value: Tensor,
        kv_cache: KVCache,
        block_table: Tensor,
        attn_mask: Optional[Tensor],
    ) -> Tensor: ...
```

This allows vLLM to support multiple attention kernels without changing the model code:

- **FlashAttention-2:** fast on H100 with FP8 support.
- **FlashInfer:** efficient for variable-length sequences.
- **PagedAttention:** vLLM's native kernel, optimised for block-sparse access.

TensorRT-LLM

NVIDIA's inference framework compiles models into optimised CUDA engines.

6.2.1 Architecture

Listing 12: TensorRT-LLM workflow

```
[PyTorch Checkpoint]
      |
      v
```

```
[Weight Converter] <- Fuses QKV, permutes heads, merges layernorm
      |
      v
[Graph Builder] <- Builds TRT engine (CUDA graphs, kernel auto-tuning)
      |
      v
[TRT Engine (.engine)] <- Serialized CUDA code + weights
      |
      v
[Triton Inference Server] <- HTTP/gRPC frontend, in-flight batching
```

Key features:

- **Engine compilation:** the graph builder runs thousands of kernel auto-tuning trials to select optimal tile sizes, block sizes, and memory layouts for the target GPU.
- **FP8 quantisation:** native FP8 tensor core support with per-tensor dynamic scaling.
- **CUDA graph capture:** entire decoding steps are captured as CUDA graphs, eliminating kernel launch overhead.
- **In-flight batching:** Triton scheduler manages request multiplexing.

6.2.2 FP8 Engine Building

Listing 13: Building an FP8 TensorRT-LLM engine

```
# Build step
trtllm-build --model_dir llama-70b \
  --dtype float16 \
  --use_fp8_kv_cache \
  --use_fp8_qdq \
  --max_batch_size 64 \
  --max_input_len 8192 \
  --max_output_len 2048 \
  --tensor_parallelism 8 \
  --output_dir engines/llama-70b-fp8/

# The --use_fp8_qdq flag enables per-tensor FP8 quantisation
# with dynamic scaling factors computed from calibration data.
```

llama.cpp

CPU-first inference with optimised kernels for x86, ARM, and GPU backends.

6.3.1 Architecture

Listing 14: llama.cpp serving stack

```
[GGUF Model File] <- Self-contained: weights + config + tokenizer
      |
      v
[GGML Runtime] <- CPU-optimized tensor library
| - Quantised GEMM (Q4, Q5, Q8)
| - SIMD kernels (AVX2, ARM NEON)
| - Metal/CUDA/Vulkan offload
      |
      v
[llama.cpp server] <- HTTP server with continuous batching
```

Key design decisions:

- **Quantised first:** all operations work on quantised data. Dequantisation happens inside the GEMM kernel, not before.
- **CPU-optimised:** uses thread pools for parallel matrix multiplication, SIMD for vector operations.
- **GPU offload:** partial GPU offload splits layers between CPU and GPU based on bandwidth and latency.
- **No dynamic shapes:** graphs are rebuilt when shapes change (less flexible than vLLM but simpler).

Comparison

Property	vLLM	TensorRT-LLM	llama.cpp	TGI
GPU support	Native	Native	Offload	Native
CPU inference	No	No	Primary	No
Input format	SafeTensors	TRT engine	GGUF	SafeTensors
Batching	Continuous	In-flight	Continuous	Continuous
PagedAttention	Native	Internal	No	No
Prefix caching	Auto	Manual	No	Auto
Quantisation	AWQ, GPTQ, FP8	FP8, INT4, INT8	GGUF 2–8 bit	AWQ, GPTQ
Kernel fusion	Module-level	Full graph	Op-level	Module-level
Model support	Broad	Optimised list	Broad	Broad

Memory Management First Principles

Why Memory Is the Bottleneck

Modern GPUs are compute-saturated for most LLM operations. The bottleneck is moving data between HBM and SRAM (shared memory).

Operation	Bottleneck	Arithmetic intensity (FLOP/byte)
Matrix multiply (large)	Compute	> 1000
Self-attention	Memory	~ 20
Autoregressive decode	Memory	~ 5
Embedding lookup	Memory	< 1

Autoregressive decoding is the most memory-bound operation in the pipeline because it processes one token at a time, loading the entire model weight for a single token's worth of computation.

Weight Caching vs. KV Caching

Two caches exist in the inference engine:

- **Weight cache:** the model parameters, loaded once and reused for all tokens. 140 GB for a 70B model in FP16.
- **KV cache:** the per-request key/value tensors, growing with sequence length. 268 GB for 100 concurrent requests at 8K context.

The weight cache is *static* (determined by model architecture). The KV cache is *dynamic* (determined by serving load). Both compete for the same HBM.

The Memory Wall

HBM capacity grows slower than model size and KV cache demand:

GPU	HBM capacity	Bandwidth
A100 80GB	80 GB	2.0 TB/s
H100 80GB	80 GB	3.35 TB/s
H200 141GB	141 GB	4.8 TB/s
B200	192 GB	8.0 TB/s

A single 70B model in FP16 consumes 140 GB, exceeding one H100. With tensor parallelism across 2 GPUs (70 GB each), only 10 GB remains per GPU for KV cache—enough for ~ 4 concurrent sequences at 8K context.

This constraint dictates the engineering trade-offs:

- Quantisation (reducing weight size increases KV cache budget).
- Block-level KV cache management (PagedAttention).
- KV cache offloading to CPU memory (swapping).
- Longer context requires more GPUs or more aggressive quantisation.

Memory-Aware Scheduling

The scheduler must track memory usage per sequence and reject requests when memory is full:

Listing 15: Memory-aware scheduling pseudocode

```
def can_admit_request(seq_len):
    """Check if we can admit a new request."""
    kv_blocks_needed = ceil(seq_len / BLOCK_SIZE)
    avail_blocks = free_pool.size()
    return kv_blocks_needed <= avail_blocks

def schedule():
    prefill = []
    decode = []
    for seq in active_sequences:
        if seq.needs_prefill:
            prefill.append(seq)
        else:
            decode.append(seq)
    # Admit new requests if capacity allows
    while queue and can_admit_request(queue[0].input_len):
        req = queue.pop(0)
        prefill.append(req)
        allocate_blocks(req, ceil(req.input_len / BLOCK_SIZE))
    return prefill, decode
```

Copy-on-Write for Parallel Sampling

When generating multiple candidate responses (beam search, best-of-N sampling), sequences initially share the same prefix. PagedAttention supports copy-on-write:

- All N sequences share the same physical blocks for the shared prefix.
- When a sequence diverges (writes a different token), a new physical block is allocated.
- The block table entry is updated for the diverging sequence; other sequences continue sharing.

This reduces memory by up to $N\times$ for the shared prefix.

Production Deployment

Architecture

A production serving system comprises multiple layers:

Listing 16: Production LLM serving architecture

```
[Load Balancer] <- Round-robin or least-connections
     |
     v
[API Servers] <- Request authentication, rate limiting, guardrailing
     |
     v
[Inference Engines] <- vLLM/TensorRT-LLM nodes
     | | |
     v v v
[GPU] [GPU] [GPU]
     |
     v
[Monitoring Stack] <- Prometheus + Grafana: TTFT, ITL, throughput, queue depth
```

Key Metrics

Definition 8.1 (Serving Metrics)**.**

- **TTFT (Time To First Token):** latency from request arrival to first output token. Dominated by pre-fill time. Target: < 500 ms.
- **ITL (Inter-Token Latency):** time between consecutive output tokens. Dominated by decode time. Target: < 30 ms/token.
- **Throughput:** output tokens per second across all requests. Target: maximise under latency SLO.
- **Queue depth:** number of pending requests. Indicates saturation.
- **Goodput:** fraction of requests meeting latency SLO. The actual metric that matters.

Load Balancing

Requests are distributed across inference nodes. The optimal strategy depends on variability:

- **Round-robin:** simplest, works when request lengths are uniform.
- **Least-connections:** routes to the node with fewest active requests. Better for variable-length requests.
- **Power-of-two choices:** pick two random nodes, route to the one with lower load. Optimal in theory.
- **Prefix-aware routing:** route requests with the same prefix to the same node, maximising prefix cache hits.

Autoscaling

Inference nodes scale based on:

- **GPU utilisation:** average HBM bandwidth usage across the node.
- **Queue depth:** sustained queue growth indicates need for more nodes.
- **P99 TTFT:** latency exceeding SLO triggers scale-out.

Cold start (launching a new inference node) takes 1–10 minutes depending on model size and whether the engine is pre-built.

Cold Start vs. Warm Models

- **Cold start:** load weights from disk, build or load engine, allocate CUDA context, warm up GPU caches. 1–10 minutes.
- **Warm model:** an already-loaded engine with cached weights. New requests are served immediately.
- **Pre-warming:** periodically send dummy requests to keep GPU caches warm and NLP kernels compiled.

Monitoring Signals

Metric	What it signals	Action
Rising TTFT	Saturation, slow pre-fill	Scale out, reduce batch size
Rising ITL	Memory bandwidth contention	Reduce concurrency, offload KV
Dropping throughput	Fragmentation, thrashing	Restart engine, tune scheduler
High queue depth	Insufficient capacity	Scale out
OOM errors	Memory leak or misconfiguration	Reduce max sequence length

Design Decision Summary

Format Design Trade-offs

Decision	Choice A	Choice B
Header encoding	Pickle (PyTorch)	JSON (SafeTensors)
Tensor alignment	None	64-byte
Quantisation storage	Separate	Embedded (GGUF)
Tokenizer	Separate file	Embedded (GGUF)
Config	Separate file	Embedded (GGUF)

Pickle maximises flexibility (arbitrary Python objects) at the cost of security and speed. SafeTensors and GGUF prioritise safety and zero-copy loading, sacrificing the ability to represent complex Python structures.

Serving Architecture Trade-offs

Decision	Choice A	Choice B
Engine format	Native PyTorch (flexible)	Compiled engine (fast)
Batching	Static (simple)	Continuous (efficient)
KV cache	Contiguous (simple)	Paged (efficient)
Kernel fusion	Module-level (portable)	Full graph (optimal)
Quantisation	Runtime (flexible)	Pre-quantised (fast)

The trend is toward more compilation (TensorRT-LLM, CUDA graphs) and more sophisticated memory management (PagedAttention, prefix caching), because memory bandwidth—not compute—is the dominant constraint.

The First-Principles Rules

1. **Memory bandwidth is the bottleneck.** Every design decision should be evaluated by its impact on HBM traffic.
2. **Avoid copies.** Zero-copy loading (mmap), in-place quantisation, and fused kernels eliminate redundant data movement.
3. **Alignment is not optional.** Unaligned memory access wastes bandwidth and prevents direct GPU loading.
4. **Fragmentation is the enemy.** Block-based allocation (PagedAttention) eliminates fragmentation at the cost of indirection.
5. **Compilation beats interpretation.** TensorRT-LLM's compiled engines outperform eager-mode PyTorch by 2–4× through kernel auto-tuning and graph optimisation.
6. **Quantise early.** Applying quantisation at the format level (GGUF, FP8 engines) enables memory-mapped loading of quantised weights, avoiding a separate dequantisation pass.

Conclusion

The path from training to serving passes through a series of format conversions, each optimising for a different constraint: safety, alignment, zero-copy loading, quantisation, and portability. The serving engine then reconstructs the model within severe memory and latency constraints, using continuous batching, paged memory management, and kernel compilation.

The key insight is that *memory is the organising constraint*. Formats are designed around memory alignment and zero-copy access. Serving engines are designed around KV cache management and memory-bound decode. Quantisation exists to trade weight fidelity for memory capacity. Every layer of the stack—from the file format to the kernel—exists to move data through the memory hierarchy efficiently.

Understanding these design decisions from first principles explains why the stack looks the way it does, and provides the framework for engineering better systems as models continue to grow.

References

[1] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023.

[2] Z. Yu et al., "SafeTensors: A Safe Serialization Format for Machine Learning Models," *HuggingFace Technical Report*, 2022.

[3] G. Gerganov et al., "GGUF: GGML Universal Format Specification," *llama.cpp*, 2023.

[4] S. Rasley et al., "DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters," *KDD*, 2020.

[5] S. Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," *SC*, 2020.

[6] NVIDIA, "TensorRT-LLM: A TensorRT Toolset for Optimizing Large Language Model Inference," *NVIDIA Developer*, 2024.

[7] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019.

[8] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *NeurIPS*, 2022.

[9] R. Pope et al., "Efficiently Scaling Transformer Inference," *MLSys*, 2023.

[10] Y. Zhong et al., "Prefix Caching for LLM Inference Serving," *arXiv:2405.04532*, 2024.